

JAVA EE

(Jakarta EE)

Les bases : architecture, fonctionnement et exemples concrets

Guide pédagogique — Servlets · JSP · EJB · JPA · CDI · JAX-RS

1. Qu'est-ce que Java EE ?

*Java EE (Java Enterprise Edition), rebaptisé **Jakarta EE** depuis 2018, est un ensemble de spécifications qui étend Java SE (Standard Edition) pour développer des applications d'entreprise : robustes, distribuées, sécurisées et multi-utilisateurs (sites web, APIs, systèmes bancaires, ERP, etc.).*

Java EE n'est pas un logiciel que l'on télécharge et qui s'exécute directement : c'est une **spécification** (un ensemble de règles et d'interfaces). Ce sont des **serveurs d'applications** (WildFly, GlassFish, Payara, TomEE...) qui implémentent ces règles et exécutent réellement le code.

Java SE vs Java EE

Critère	Java SE	Java EE
Usage	Applications de bureau, scripts, applications simples	Applications d'entreprise, web, distribuées
Contenu	JVM, classes de base (String, Collections, I/O...)	Java SE + API supplémentaires (Servlet, EJB, JPA, JMS...)
Exécution	N'importe quelle JVM	Nécessite un serveur d'applications compatible
Exemples d'usage	Application console, jeu simple, utilitaire	Site e-commerce, système bancaire, ERP

À savoir : en 2018, Oracle a transféré Java EE à la fondation Eclipse. Pour des raisons de marque, le nom a changé en **Jakarta EE**. Les concepts restent identiques ; seul le préfixe des packages a changé progressivement de `javax.*` vers `jakarta.*`.

2. Architecture générale

Une application Java EE est organisée en **couches (tiers)** : chaque couche a une responsabilité précise, ce qui facilite la maintenance et la répartition du travail sur plusieurs machines.

Architecture multi-couches de Java EE

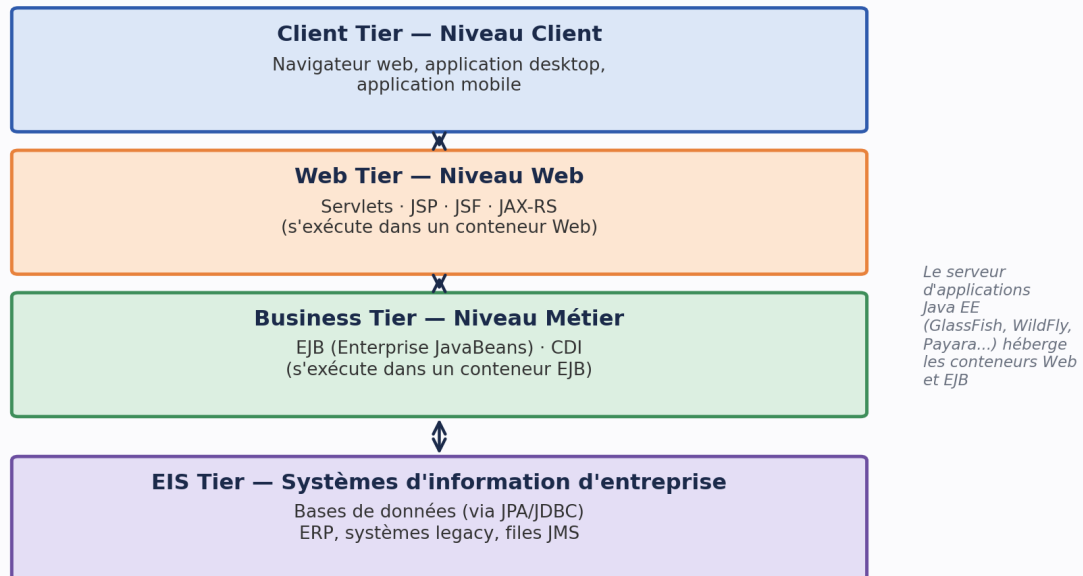


Figure 1 — Les quatre couches d'une application Java EE

Le rôle de chaque couche

- **Client Tier** — L'interface avec laquelle l'utilisateur interagit : navigateur web, application mobile ou logicielle desktop.
- **Web Tier** — Reçoit les requêtes HTTP, les traite via des Servlets/JSP/JSF et renvoie une réponse (page HTML, JSON...).
- **Business Tier** — Contient la logique métier de l'application (calculs, règles de gestion) via des composants EJB.
- **EIS Tier** — Le niveau des systèmes d'information d'entreprise : bases de données, ERP, systèmes existants (legacy).

Le **serveur d'applications** (aussi appelé conteneur JEE) héberge et gère le cycle de vie des composants Web et EJB : il s'occupe des transactions, de la sécurité, de la concurrence et de l'injection de dépendances, ce qui allège considérablement le travail du développeur.

3. Comment ça marche ? Le cycle d'une requête

Voici ce qui se passe concrètement, étape par étape, lorsqu'un utilisateur clique sur un lien ou soumet un formulaire dans une application Java EE classique :

Cycle de vie d'une requête dans une application Java EE

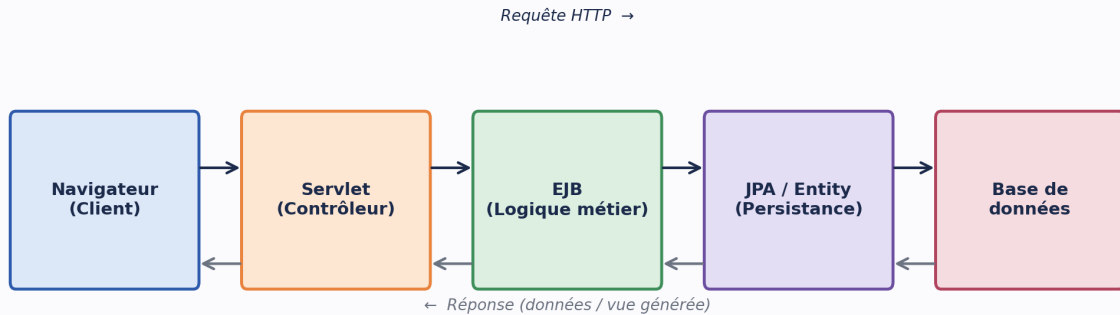


Figure 2 — Trajet d'une requête HTTP à travers les couches

1. Le **navigateur** envoie une requête HTTP (ex. GET /produits) vers le serveur.
2. Le **Servlet** (contrôleur) reçoit la requête, l'analyse et décide de l'action à réaliser.
3. Le Servlet appelle un **EJB** qui exécute la logique métier (ex. calculer un prix, vérifier un stock).
4. L'EJB utilise **JPA** pour lire ou écrire des données dans la **base de données**.
5. Le résultat remonte : l'EJB renvoie les données au Servlet, qui les transmet à une page **JSP** pour générer le HTML final envoyé au navigateur.

4. Les principales technologies de Java EE

Principales API / spécifications de Java EE

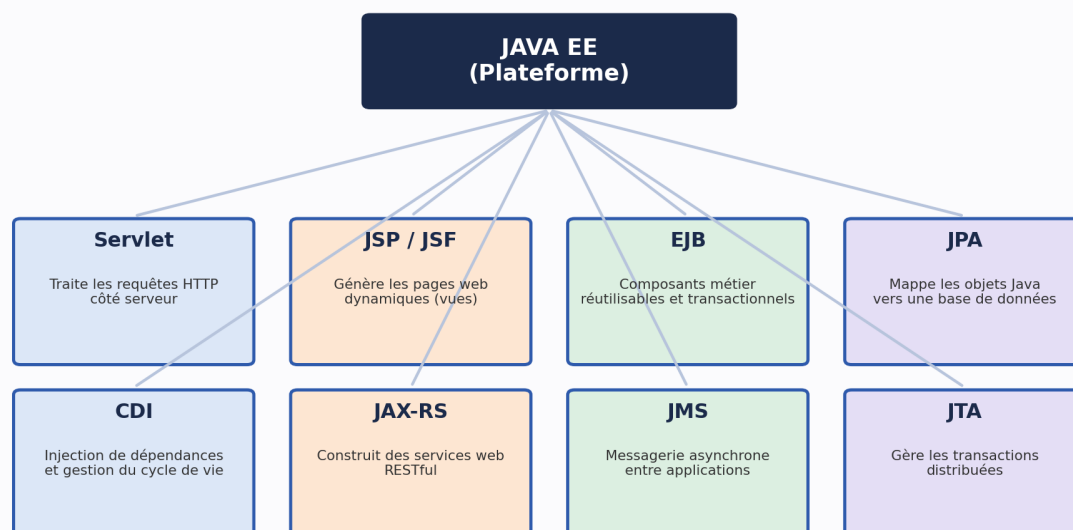


Figure 3 — Les API/spécifications les plus utilisées

4.1 — Les Servlets

Définition : un Servlet est une classe Java qui s'exécute côté serveur et qui traite les requêtes HTTP (GET, POST...) envoyées par un client. C'est le point d'entrée classique d'une application web Java EE — l'équivalent d'un « contrôleur » dans le modèle MVC.

Exemple : un Servlet simple qui répond « Bonjour »

```
@WebServlet("/bonjour")
public class BonjourServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html;charset=UTF-8");
        PrintWriter out = response.getWriter();
        out.println("<h1>Bonjour depuis un Servlet !</h1>");
    }
}
```

L'annotation `@WebServlet` associe l'URL `/bonjour` à cette classe. La méthode `doGet()` est appelée automatiquement à chaque requête GET sur cette URL.

4.2 — JSP (JavaServer Pages)

Définition : JSP permet d'écrire des pages HTML contenant du code Java intégré, afin de générer des pages web dynamiques (dont le contenu change selon les données). JSP est souvent utilisée comme vue dans le modèle MVC, pendant que le Servlet joue le rôle de contrôleur.

Exemple : afficher une liste de produits

```
<%@ page contentType="text/html;charset=UTF-8" %>
<html>
<body>
  <h2>Liste des produits</h2>
  <ul>
    <c:forEach var="produit" items="${listeProduits}">
      <li>${produit.nom} - ${produit.prix} MAD</li>
    </c:forEach>
  </ul>
</body>
</html>
```

La balise `<c:forEach>` (JSTL) parcourt la liste `listeProduits` envoyée par le Servlet et affiche chaque produit dynamiquement.

4.3 — EJB (Enterprise JavaBeans)

Définition : un EJB est un composant Java qui encapsule la logique métier de l'application. Le conteneur EJB gère automatiquement les transactions, la sécurité et la concurrence. Il existe deux grands types : les **Session Beans** (logique métier) et les **Message-Driven Beans** (traitement de messages asynchrones).

Exemple : un Session Bean de gestion de commandes

```
@Stateless
public class CommandeService {

    @PersistenceContext
    private EntityManager em;

    public void passerCommande(Commande commande) {
        commande.setStatut("EN_COURS");
        em.persist(commande); // enregistre la commande en base
    }

    public List<Commande> listerCommandes() {
        return em.createQuery(
            "SELECT c FROM Commande c", Commande.class
        ).getResultList();
    }
}
```

@Stateless indique un EJB sans état conservé entre deux appels : idéal pour des opérations métier ponctuelles comme celle-ci.

4.4 — JPA (Java Persistence API)

Définition : JPA est une API qui permet de faire correspondre (« mapper ») des objets Java à des tables de base de données, sans écrire de SQL manuellement. On parle d'**ORM** (Object-Relational Mapping). Chaque classe Java annotée @Entity correspond à une table.

Exemple : une entité JPA « Produit »

```
@Entity
@Table(name = "produits")
public class Produit {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String nom;
    private double prix;

    // getters et setters
}
```

Avec cette annotation, JPA crée automatiquement une table produits avec les colonnes correspondant aux attributs de la classe, et permet de la manipuler via l'objet Java.

4.5 — CDI (Contexts and Dependency Injection)

Définition : CDI est le mécanisme d'injection de dépendances de Java EE. Plutôt que de créer manuellement les objets dont une classe a besoin (`new MaClasse()`), on laisse le conteneur les « injecter » automatiquement grâce à l'annotation `@Inject`.

Exemple : injection d'un service dans un Servlet

```
@WebServlet("/commande")
public class CommandeServlet extends HttpServlet {

    @Inject
    private CommandeService commandeService;    // injecté par CDI

    @Override
    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response) {
        commandeService.passerCommande(new Commande());
    }
}
```

Le conteneur crée et fournit lui-même l'instance de `CommandeService` : le code est plus simple, plus testable et plus faiblement couplé.

4.6 — JAX-RS (services web REST)

Définition : JAX-RS est l'API Java EE pour créer des **services web RESTful**, c'est-à-dire des APIs qui échangent des données (souvent au format JSON) via des méthodes HTTP standard (GET, POST, PUT, DELETE). Très utilisée aujourd'hui pour connecter une application front-end (React, Angular...) à un backend Java.

Exemple : une API REST pour les produits

```
@Path("/produits")
public class ProduitResource {

    @Inject
    private ProduitService produitService;

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public List<Produit> getTousLesProduits() {
        return produitService.listerProduits();
    }

    @POST
    @Consumes(MediaType.APPLICATION_JSON)
    public Response ajouterProduit(Produit produit) {
        produitService.ajouter(produit);
        return Response.status(201).build();
    }
}
```

Un appel GET sur `/produits` renverra automatiquement la liste des produits au format JSON, prête à être consommée par une application front-end.

5. Exemple complet : comment les briques s'assemblent

Reprenons l'exemple d'une petite application de gestion de commandes pour voir comment toutes les technologies vues précédemment collaborent :

Étape	Technologie	Rôle dans l'exemple
1	Servlet / JAX-RS	Reçoit la requête HTTP du client (ex. « créer une commande »)
2	CDI (@Inject)	Injecte automatiquement le service métier nécessaire
3	EJB (@Stateless)	Exécute la logique métier : validation, calcul du total...
4	JPA (@Entity)	Enregistre la commande dans la base de données
5	JSP ou JSON	Renvoie une confirmation à l'utilisateur (page HTML ou réponse API)

À retenir : chaque technologie a une responsabilité unique (principe de *séparation des préoccupations*). C'est ce qui rend les applications Java EE faciles à maintenir et à faire évoluer, même à grande échelle.

6. Les serveurs d'applications

Pour exécuter une application Java EE, il faut la déployer sur un **serveur d'applications** compatible avec les spécifications Jakarta EE. Voici les plus utilisés :

Serveur	Particularité
WildFly	Serveur open-source de Red Hat, très populaire, rapide à configurer
GlassFish	Implémentation de référence, maintenue par Eclipse Foundation
Payara Server	Basé sur GlassFish, orienté production et cloud
Apache TomEE	Version « Java EE » de Tomcat, légère et modulaire
IBM WebSphere	Solution d'entreprise, souvent utilisée en environnement bancaire

Comment se déploie une application ?

1. Le projet est compilé et empaqueté sous la forme d'un fichier **.war** (Web Archive) ou **.ear** (Enterprise Archive).
2. Ce fichier est déposé (« déployé ») dans le dossier de déploiement du serveur d'applications.
3. Le serveur lit les annotations et fichiers de configuration, crée les conteneurs nécessaires et démarre l'application.
4. L'application devient accessible via une URL, généralement du type `http://localhost:8080/mon-application`.

7. Glossaire des termes essentiels

- **API** — Interface de programmation : un ensemble de règles permettant à des programmes de communiquer entre eux.
- **Conteneur (Container)** — Environnement d'exécution géré par le serveur d'applications, qui prend en charge le cycle de vie des composants (Servlets, EJB...).
- **ORM** — Object-Relational Mapping : technique qui fait correspondre des objets Java à des lignes de tables en base de données (implémentée par JPA).
- **Annotation** — Métadonnée placée au-dessus d'une classe ou d'une méthode (ex. `@Entity`, `@Stateless`) qui indique au conteneur comment traiter cet élément.
- **MVC** — Modèle-Vue-Contrôleur : un patron de conception qui sépare les données (Modèle), l'affichage (Vue) et la logique de traitement (Contrôleur).
- **Injection de dépendances** — Technique où le conteneur fournit automatiquement à un objet les autres objets dont il a besoin, plutôt que de les créer lui-même.
- **WAR / EAR** — Formats d'archive (fichiers compressés) utilisés pour emballer et déployer une application Java EE sur un serveur.
- **REST** — Style d'architecture pour construire des APIs web basées sur les méthodes HTTP standard (GET, POST, PUT, DELETE).

8. La sécurité en Java EE

Définition : la sécurité Java EE repose sur deux notions clés : l'**authentification** (vérifier qui est l'utilisateur, via un login/mot de passe par exemple) et l'**autorisation** (vérifier ce que cet utilisateur a le droit de faire, en fonction de son rôle).

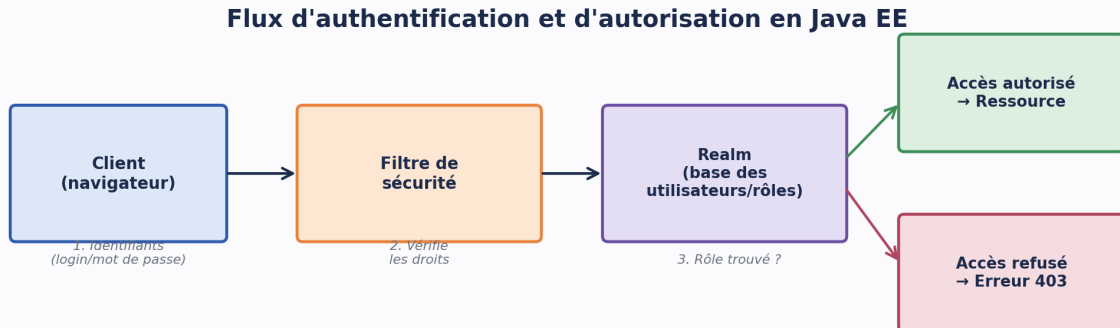


Figure 4 — Flux d'authentification et d'autorisation

Le conteneur (serveur d'applications) s'occupe de tout ce processus automatiquement : le développeur n'a qu'à déclarer les rôles autorisés sur chaque ressource, sans écrire lui-même la logique de vérification.

Exemple : protéger un Servlet par rôle

```

@WebServlet("/admin")
@WebServletSecurity(
    @HttpConstraint(rolesAllowed = {"ADMINISTRATEUR"})
)
public class AdminServlet extends HttpServlet {

    @Override
    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws IOException {
        response.getWriter().println("Espace réservé aux administrateurs");
    }
}
    
```

Seuls les utilisateurs authentifiés possédant le rôle ADMINISTRATEUR pourront accéder à cette page ; les autres reçoivent automatiquement une erreur 403 (accès refusé).

Exemple : récupérer l'utilisateur connecté dans le code

```
@Inject
private SecurityContext securityContext;

public void afficherUtilisateur() {
    Principal utilisateur = securityContext.getCallerPrincipal();
    boolean estAdmin = securityContext.isCallerInRole("ADMINISTRATEUR");
    System.out.println("Connecté : " + utilisateur.getName());
}
```

9. Les transactions avec JTA

Définition : une transaction regroupe plusieurs opérations qui doivent réussir ou échouer **toutes ensemble** (principe ACID). JTA (Java Transaction API) est le mécanisme de Java EE qui gère ces transactions, notamment lorsqu'elles impliquent plusieurs sources de données.

Par défaut, chaque méthode d'un EJB est exécutée dans une transaction gérée automatiquement par le conteneur (**CMT — Container-Managed Transaction**) : si une exception survient, toutes les modifications sont annulées (*rollback*).

Exemple : transfert d'argent entre deux comptes

```
@Stateless
public class VirementService {

    @PersistenceContext
    private EntityManager em;

    @TransactionAttribute(TransactionAttributeType.REQUIRED)
    public void virer(Long idSource, Long idDestination, double montant) {
        Compte source = em.find(Compte.class, idSource);
        Compte destination = em.find(Compte.class, idDestination);

        source.debiter(montant);          // si le solde est insuffisant,
        destination.crediter(montant);    // une exception annule TOUT le virement
    }
}
```

Si `debiter()` lève une exception (solde insuffisant), la méthode `crediter()` ne sera jamais validée en base : la transaction est annulée dans son intégralité, ce qui évite de créer de l'argent « from nowhere ».

À savoir : il existe aussi les transactions gérées par le développeur lui-même (**BMT — Bean-Managed Transaction**), utiles pour les cas où l'on souhaite contrôler précisément le début et la fin d'une transaction dans le code.

10. JMS : la messagerie asynchrone

Définition : JMS (Java Message Service) permet à des composants d'échanger des messages **sans être connectés directement ni actifs au même moment**. Un **producteur** dépose un message dans une **file**, et un **consommateur** le récupère quand il est prêt. Très utilisé pour découpler des traitements lourds (envoi d'e-mails, génération de factures...).

Messagerie asynchrone avec JMS



Le producteur et le consommateur n'ont pas besoin d'être actifs en même temps : la file conserve les messages (couplage faible).

Figure 5 — Producteur, file JMS et consommateur

Exemple : envoyer un message

```

@Inject
private JMSContext contexte;

@Resource(lookup = "jms/FileCommandes")
private Queue fileCommandes;

public void envoyerNotification(String contenu) {
    contexte.createProducer().send(fileCommandes, contenu);
}
  
```

Exemple : consommer un message (Message-Driven Bean)

```

@MessageDriven(activationConfig = {
    @ActivationConfigProperty(propertyName = "destinationLookup",
        propertyValue = "jms/FileCommandes")
})
public class NotificationConsumer implements MessageListener {

    @Override
    public void onMessage(Message message) {
        try {
            String contenu = ((TextMessage) message).getText();
            System.out.println("Notification reçue : " + contenu);
        } catch (JMSEException e) {
            e.printStackTrace();
        }
    }
}
  
```

Le Message-Driven Bean est automatiquement réveillé par le conteneur dès qu'un message arrive dans la file : aucun code d'attente actif n'est nécessaire.

11. Bean Validation : valider les données

Définition : Bean Validation est l'API Java EE qui permet de vérifier automatiquement la validité des données d'un objet (champs obligatoires, formats, longueurs...) grâce à de simples annotations, sans écrire de code de vérification manuel.

Exemple : valider une entité « Utilisateur »

```
public class Utilisateur {  
  
    @NotNull(message = "Le nom est obligatoire")  
    @Size(min = 2, max = 50)  
    private String nom;  
  
    @Email(message = "L'adresse email n'est pas valide")  
    private String email;  
  
    @Min(value = 18, message = "L'utilisateur doit être majeur")  
    private int age;  
  
}
```

Exemple : déclencher la validation

```
@Inject  
private Validator validator;  
  
public void inscrire(Utilisateur utilisateur) {  
    Set<ConstraintViolation<Utilisateur>> erreurs =  
        validator.validate(utilisateur);  
  
    if (!erreurs.isEmpty()) {  
        throw new ValidationException(erreurs.toString());  
    }  
    // sinon, l'utilisateur est valide : on peut l'enregistrer  
}
```

Dans une API JAX-RS, il suffit d'ajouter l'annotation `@Valid` devant un paramètre pour que la validation se déclenche automatiquement avant l'exécution de la méthode.

12. Java EE face à Spring : quelques repères

Spring est un autre framework très répandu pour développer des applications Java d'entreprise. Il n'est pas une alternative « opposée » à Java EE, mais une approche différente — de nombreux projets Spring s'appuient d'ailleurs sur des spécifications Java EE (JPA, Bean Validation, Servlet...).

Aspect	Java EE / Jakarta EE	Spring
Nature	Spécification standard implémentée par plusieurs serveurs	Framework concret, un seul éditeur principal (VMware)
Configuration	Annotations + serveur d'applications complet	Spring Boot : configuration minimale, serveur embarqué
Portabilité	Le code fonctionne sur tout serveur compatible Jakarta EE	Dépend des bibliothèques Spring choisies
Courbe d'apprentissage	Standardisée, proche des spécifications officielles	Très riche en outils, écosystème très large

En résumé : Java EE mise sur la standardisation et la portabilité entre serveurs ; Spring mise sur la flexibilité et un écosystème très outillé. Les concepts appris ici (Servlet, JPA, injection de dépendances, transactions) restent valables et se retrouvent dans les deux mondes.

13. Structure type d'un projet Java EE (Maven)

La plupart des projets Java EE utilisent **Maven** pour gérer les dépendances et la construction du projet. Voici l'arborescence typique d'une application web Java EE :

```
mon-application/
|-- pom.xml                # dépendances et configuration Maven
`-- src/
    |-- main/
        |-- java/
            |-- com/exemple/app/
                |-- servlet/
                    |-- CommandeServlet.java
                |-- ejb/
                    |-- CommandeService.java
                |-- entity/
                    |-- Commande.java
                |-- rest/
                    |-- ProduitResource.java
        |-- resources/
            |-- META-INF/
                |-- persistence.xml # configuration JPA
    |-- webapp/
        |-- WEB-INF/
            |-- web.xml            # configuration du module web
            |-- beans.xml          # active CDI
        |-- index.jsp
```

Une fois compilé avec `mvn package`, ce projet génère un fichier `.war` prêt à être déployé sur un serveur d'applications comme WildFly ou Payara.

Pour aller plus loin : la documentation officielle de la fondation Eclipse (jakarta.ee) et les tutoriels du serveur WildFly sont d'excellents points de départ pour pratiquer avec un vrai projet.